ElderMeds Disease Risk Prediction Algorithms

Generated: 2026-08-24

This report documents the algorithms used for the diabetes, stroke, and hypertension risk prediction modules. The values come from the saved model metadata in ml-service/app/models and the selection logic in the training scripts.

Executive Summary

- Diabetes: selected algorithm = XGBoost.
- Stroke: selected algorithm = Logistic Regression.
- Hypertension: selected algorithm = Decision Tree.

All three modules train the same four candidate algorithm families and save the selected model plus the preprocessing object. At runtime the inference service transforms user inputs, calls predict_proba, converts the positive-class probability into Low, Medium, or High risk, and returns the selectedAlgorithm field from metadata.

Diabetes Risk Prediction

Selected algorithm: XGBoost

XGBoost was selected because it ranked highest under the diabetes ranking rule. It had the best recall, F1-score, ROC-AUC, and accuracy among the tested candidates.

Selected Model Values

- accuracy: 0.7427
- precision: 0.7195
- recall: 0.7954
- f1Score: 0.7556
- rocAuc: 0.8204
- confusionMatrix: [[4878, 2192], [1446, 5623]]
- Confusion matrix format: [[true negative, false positive], [false negative, true positive]].
- class balance in training set: positive=28277, negative=28276

How The Algorithm Was Chosen

- Candidate algorithms: Logistic Regression, Decision Tree, Random Forest, and XGBoost.
- Train/test split: 80/20 stratified split with random_state=42.
- Ranking rule: Recall, then F1-score, then ROC-AUC, then accuracy.
- The training code gives recall first priority so the selected model favors finding positive disease-risk cases before optimizing the secondary scores.

Comparison Values

```
Algorithm               Acc    Prec  Recall      F1     ROC  PosRate     Confusion Matrix
-------------------------------------------------------------------------------------------
Logistic Regression   0.7370  0.7280  0.7568  0.7421  0.8127   0.5020 [[5071, 1999], [1719, 5350]]
Decision Tree         0.7286  0.7179  0.7531  0.7351  0.8022   0.4990 [[4978, 2092], [1745, 5324]]
Random Forest         0.7047  0.6983  0.7208  0.7094  0.7700   0.4995 [[4869, 2201], [1974, 5095]]
XGBoost               0.7427  0.7195  0.7954  0.7556  0.8204   0.5014 [[4878, 2192], [1446, 5623]]
```

Risk Level Conversion

- ML probability < 0.40 -> Low
- 0.40 <= ML probability < 0.70 -> Medium
- ML probability >= 0.70 -> High
- Clinical guardrail can raise the final risk when blood sugar, BMI, high BP, family history, or low activity are concerning.
- The service returns riskType, riskLevel, confidence, selectedAlgorithm, factors, probability, featuresUsed, and derived values.

Input Features

- Age

- Sex
- BMI
- HighBP
- Smoker
- PhysActivity
- HeartDiseaseorAttack
- Stroke
- GenHlth
- DiffWalk

Candidate Model Settings

- Logistic Regression: max_iter=2000, class_weight=balanced, random_state=42
- Decision Tree: class_weight=balanced, min_samples_leaf=20, random_state=42
- Random Forest: n_estimators=350, class_weight=balanced_subsample, random_state=42
- XGBoost: n_estimators=350, learning_rate=0.05, max_depth=5, subsample=0.9, colsample_bytree=0.9, scale_pos_weight=trainNegative/trainPositive

Files And Dataset

- Dataset: ml-service/data/raw/diabetes_binary_5050split_health_indicators_BRFSS2015.csv
- Saved model: ml-service/app/models/diabetes_model.pkl
- Saved preprocessor: ml-service/app/models/diabetes_preprocessor.pkl
- Training script: ml-service/training/diabetes/train_diabetes_model.py
- Inference script: ml-service/app/services/diabetes_inference.py

Stroke Risk Prediction

Selected algorithm: Logistic Regression

Logistic Regression was selected because it achieved the highest recall. Even though another model had a higher F1-score, the ranking rule chooses recall first for screening safety.

Selected Model Values

- accuracy: 0.7466
- precision: 0.1384
- recall: 0.8000
- f1Score: 0.2360
- rocAuc: 0.8426
- confusionMatrix: [[723, 249], [10, 40]]
- Confusion matrix format: [[true negative, false positive], [false negative, true positive]].
- class balance in training set: positive=199, negative=3889

How The Algorithm Was Chosen

- Candidate algorithms: Logistic Regression, Decision Tree, Random Forest, and XGBoost.
- Train/test split: 80/20 stratified split with random_state=42.
- Ranking rule: Recall, then F1-score, then ROC-AUC, then accuracy.
- The code comment says stroke screening prioritizes recall because missing a true stroke-risk case is costly.

Comparison Values

```
Algorithm               Acc    Prec  Recall      F1     ROC  PosRate     Confusion Matrix
-------------------------------------------------------------------------------------------
Logistic Regression   0.7466  0.1384  0.8000  0.2360  0.8426   0.3189 [[723, 249], [10, 40]]
Decision Tree         0.7906  0.1306  0.5800  0.2132  0.7084   0.1688 [[779, 193], [21, 29]]
Random Forest         0.9432  0.3462  0.1800  0.2368  0.7929   0.1210 [[955, 17], [41, 9]]
XGBoost               0.8650  0.2027  0.6000  0.3030  0.8252   0.1793 [[854, 118], [20, 30]]
```

Risk Level Conversion

- ML probability < 0.30 -> Low
- 0.30 <= ML probability < 0.65 -> Medium
- ML probability >= 0.65 -> High
- The service returns riskType, riskLevel, confidence, selectedAlgorithm, factors, probability,

featuresUsed, and derived values.

Input Features

- age
- hypertension
- heart_disease
- avg_glucose_level
- bmi
- gender
- ever_married
- work_type
- Residence_type
- smoking_status

Candidate Model Settings

- Logistic Regression: max_iter=2000, class_weight=balanced, random_state=42
- Decision Tree: class_weight=balanced, min_samples_leaf=15, random_state=42
- Random Forest: n_estimators=450, class_weight=balanced_subsample, min_samples_leaf=3, random_state=42
- XGBoost: n_estimators=350, learning_rate=0.04, max_depth=4, subsample=0.9, colsample_bytree=0.9, scale_pos_weight=trainNegative/trainPositive

Files And Dataset

- Dataset: ml-service/data/raw/healthcare-dataset-stroke-data.csv
- Saved model: ml-service/app/models/stroke_model.pkl
- Saved preprocessor: ml-service/app/models/stroke_preprocessor.pkl
- Training script: ml-service/training/stroke/train_stroke_model.py
- Inference script: ml-service/app/services/stroke_inference.py

Hypertension Risk Prediction

Selected algorithm: Decision Tree

Decision Tree was selected because it had the highest balanced accuracy. This mattered because a high ordinary accuracy model could still be weak if it mostly predicted one class.

Selected Model Values

- accuracy: 0.5205
- balancedAccuracy: 0.5049
- precision: 0.7226
- recall: 0.5405
- f1Score: 0.6184
- rocAuc: 0.5031
- confusionMatrix: [[4619, 5221], [11560, 13597]]
- Confusion matrix format: [[true negative, false positive], [false negative, true positive]].
- class balance in training set: positive=100624, negative=39361

How The Algorithm Was Chosen

- Candidate algorithms: Logistic Regression, Decision Tree, Random Forest, and XGBoost.
- Train/test split: 80/20 stratified split with random_state=42.
- Ranking rule: Balanced accuracy, then ROC-AUC, then F1-score, then recall, then accuracy.
- The code explicitly avoids selecting a model that simply predicts the majority class. Balanced accuracy is therefore the first ranking metric.

Comparison Values

```
Algorithm               Acc  BalAcc    Prec  Recall      F1     ROC  PosRate     Confusion Matrix
-------------------------------------------------------------------------------------------------
Logistic Regression   0.4985  0.4949  0.7147  0.5032  0.5906  0.4954   0.5000 [[4787, 5053], [12497, 12660]]
Decision Tree         0.5205  0.5049  0.7226  0.5405  0.6184  0.5031   0.5305 [[4619, 5221], [11560, 13597]]
Random Forest         0.7188  0.5000  0.7188  1.0000  0.8364  0.5033   0.6135 [[0, 9840], [1, 25156]]
XGBoost               0.5291  0.5021  0.7203  0.5639  0.6326  0.5020   0.5048 [[4332, 5508], [10972, 14185]]
```

Risk Level Conversion

- ML probability < 0.40 -> Low
- 0.40 <= ML probability < 0.70 -> Medium
- ML probability >= 0.70 -> High
- The service returns riskType, riskLevel, confidence, selectedAlgorithm, factors, probability, featuresUsed, and derived values.

Input Features

- Age
- BMI
- Cholesterol
- Systolic_BP
- Diastolic_BP
- Alcohol_Intake
- Stress_Level
- Salt_Intake
- Sleep_Duration
- Heart_Rate
- LDL
- HDL
- Triglycerides
- Glucose
- Country
- Smoking_Status
- Physical_Activity_Level
- Family_History
- Diabetes
- Gender
- Education_Level
- Employment_Status

Default Values Used By Inference

- Age: 54.0000
- BMI: 27.5000
- Cholesterol: 225.0000
- Systolic_BP: 135.0000
- Diastolic_BP: 89.0000
- Alcohol_Intake: 15.0000
- Stress_Level: 5.0000
- Salt_Intake: 8.5000
- Sleep_Duration: 7.0000
- Heart_Rate: 75.0000
- LDL: 130.0000
- HDL: 65.0000
- Triglycerides: 150.0000
- Glucose: 134.0000
- Country: Saudi Arabia
- Smoking_Status: Current
- Physical_Activity_Level: Low
- Family_History: No
- Diabetes: No
- Gender: Female
- Education_Level: Secondary
- Employment_Status: Retired

Candidate Model Settings

- Logistic Regression: max_iter=2000, class_weight=balanced, random_state=42

- Decision Tree: class_weight=balanced, min_samples_leaf=40, random_state=42
- Random Forest: n_estimators=250, class_weight=balanced_subsample, min_samples_leaf=5, random_state=42
- XGBoost: n_estimators=250, learning_rate=0.05, max_depth=5, subsample=0.9, colsample_bytree=0.9, scale_pos_weight=trainNegative/trainPositive

Files And Dataset

- Dataset: ml-service/data/raw/hypertension_dataset.csv
- Saved model: ml-service/app/models/hypertension_model.pkl
- Saved preprocessor: ml-service/app/models/hypertension_preprocessor.pkl
- Training script: ml-service/training/hypertension/train_hypertension_model.py
- Inference script: ml-service/app/services/hypertension_inference.py

Conclusion

The final algorithm choices are evidence-based rather than arbitrary. Diabetes uses XGBoost because it gave the strongest overall metrics under the recall-first ranking rule. Stroke uses Logistic Regression because recall was prioritized for screening safety and it detected the largest share of true risk cases. Hypertension uses Decision Tree because balanced accuracy was prioritized to avoid a model that mostly predicts the majority class.

These results should be described as risk awareness or screening support, not a medical diagnosis. The outputs depend on the dataset quality, feature mapping, threshold choices, and the user’s entered values.